Jsp - Servlet

Chapitre 4: Génération de la réponse

Dr. Ahmad Khoureich Ka

UADB

L3 D2AW

1. Structure de la réponse
2. Les codes de statut
3. Les en-têtes HTTP de réponse
4. Inclusion de réponse

Structure de la réponse

La réponse générée par le serveur pour le client est composée de 3 parties :

1. **Une ligne de statut**
   contenant la version du protocole HTTP, un code de statut et un petit message descriptif de ce dernier.

2. **Un certain nombre d'en-têtes HTTP de réponse**
   indiquant au client comment traiter la réponse ;

3. **Le corps de la réponse**,
   ce qui est affiché dans le navigateur.

Structure de la réponse

Exemple :

```
HTTP/1.1 200 OK

Content-Type: text/html
en-tête2: ...
...
en-têteN: ...

<!DOCTYPE html>
<html>
<head>...</head>
<body>
...
</body>
</html>
```

Les codes de statut

- Les codes de statut sont utilisés par le serveur pour informer le client de l'état de la réponse qu'il s'apprête à envoyer.
- Les codes de statut sont des nombres entiers. L'interface `HttpServletResponse` les définit comme des constantes.

Exemple :

- `200` : la requête traitée avec succès, la réponse est entrain d'être envoyée.
- `404` : le document est introuvable.
- `500` : le serveur a rencontré une erreur lors du taitement de la requête.

Les codes de statut

- Un code de statut est un nombre entier de 3 chiffres dont le premier indique sa catégorie.

- Il y'a 5 catégories de code de statut.

1xx	Information à l'attention du client.
2xx	Indique que le traitement de la requête a réussi.
3xx	Indique qu'une redirection est nécessaire parce que la ressource a été déplacée.
4xx	Indique une erreur du client dans la formulation de la requête.
5xx	Indique qu'une erreur est survenue au niveau du serveur lors du traitement de la requête.

Les codes de statut les plus courants

Nom de la constante	Code	Signification
SC_CONTINUE	100	Indique au client qu'il peut poursuivre et envoyer le corps de la requête.
SC_OK	200	La génération de la réponse a reussi.
SC_NO_CONTENT	204	Le serveur a traité la requête mais il n'a rien à répondre. Le navigateur reste sur la page affichée.
SC_MOVED_PERMANENTLY	301	La ressource demandée a changé d'emplacement. Une nouvelle url est envoyée avec l'en-tête Location.
SC_FOUND	302	La ressource demandée a temporairement changé d'emplacement. mais le client peut continuer à utiliser la même url. Redirection automatique vers le nouvel emplacement.
SC_NOT_MODIFIED	304	La réponse n'est pas différente de la précédente. La navigateur doit continuer à afficher la même page.
SC_UNAUTHORIZED	401	La requête ne contient pas les paramètres d'authentification nécessaires pour accéder à la ressource.
SC_NOT_FOUND	404	La ressource demandée n'est pas trouvée.
SC_INTERNAL_SERVER_ERROR	500	Une erreur est survenu au niveau du serveur donc la requête ne peut pas être traitée
SC_SERVICE_UNAVAILABLE	503	Le serveur est temporairement indisponible (il est débordé ou en maintenance).

Définition d'un code de statut

- La définition du code de statut d'une réponse se fait avec la méthode (de **`HttpServletResponse`**) :

```
public void setStatus(int code)
```

Remarques

- L'argument de cette méthode est le nombre entier associé au code de statut, mais il est préférable d'utiliser la constante associée définie par l'interface **`HttpServletResponse`**.

  Exemple :

```
response.setStatus(HttpServletResponse.SC_SERVICE_UNAVAILABLE);
// est préférée à
response.setStatus(503);
```

- Lorsque le code de statut n'est pas explicitement défini pour la réponse, la valeur par défaut **`200 (SC_OK)`** est utilisée.

Définition d'un code de statut

Code de statut `SC_NOT_MODIFIED`

- Le code de statut `SC_NOT_MODIFIED` (304) indique au navigateur que la réponse précédemment reçue n'est pas différente de celle demandée.
- La navigateur affiche la page qui est dans le cache.

Les codes de statut de redirection `301` et `302`

- L'utilisation des codes de statut `301` et `302` nécessite la définition d'une url localisant la ressource. Pour cela, il faut après la définition du code de statut présicer la valeur de l'en-tête `"Location"`.

```
response.setStatus(HttpServletResponse.SC_FOUND);
response.setHeader("Location", "url");
```

Définition d'un code de statut

Remarque :

- pour les codes de statut de redirection il est préférable d'utiliser la méthode :

```
public void sendRedirect(String url)
```

qui remplace les 2 instructions précédentes.

Remarques sur sendRedirect :

1. Pour tenir compte de la session, les urls données à `sendRedirect` doivent être encodées avec la méthode :

```
public String encodeRedirectURL(String url)
```

de l'objet `HttpServletResponse`.

Exemple :

```
response.sendRedirect(response.encodeRedirectURL(url));
```

Définition d'un code de statut

1. L'url donnée en argument à la méthode **`sendRedirect`** peut être :
   - relative à la requête courante
     exemple : **`url_page`**,
   - relative à la racine du conteneur de servlets
     exemple : **`/context/url_page`**,
   - un identifiant uniforme de ressource
     exemple : **`//uadb.edu.sn`**.

Définition d'un code de statut

Les codes de statut d'erreur

- Pour les codes de statut représentant une erreur comme **404 (NOT FOUND)**, la méthode :

```
public void sendError(int sc, String message)
```

de l'objet **HttpServletResponse** est préférable à **setStatus**.

- En plus de définir le code d'erreur, **sendError** permet d'envoyer un message explicatif au client.

Exemple :

```
response.sendError(HttpServletResponse.SC_NOT_FOUND, "La page que
    vous demandez est introuvable!");
```

Les en-têtes HTTP de réponse

- Des en-têtes peuvent être définis pour fournir au client des informations additionnelles sur la réponse attendue : ce sont les en-têtes HTTP de réponse.

  Exemple définir :
  - le type de contenu de la réponse ;
  - la durée de validité de la réponse ;
  - le nouvel emplacement de la ressource demandée ;
  - les directives de mise en cache ;
  - la définition des cookies.

Définitions des en-têtes HTTP de réponse

- La définition des en-têtes HTTP de réponse se fait à l'aide de méthodes de **`HttpServletResponse`** :
  - pour n'importe quel type d'en-tête :

```
public void setHeader(String nomEntete, String valeurEntete)
```

  - pour les en-têtes dont la valeur est un nombre entier :

```
public void setIntHeader(String nomEntete, int value)
```

  - pour les en-têtes dont la valeur est une **`Date`** :

```
public void setDateHeader(String nomEntete, long date)
```

    L'argument **`date`** est spécifié en nombre de millisecondes (écoulé depuis le 1 janv. 1970).

Définitions des en-têtes HTTP de réponse

Remarques

- Si l'en-tête que défini l'une des méthodes `setHeader`, `setIntHeader` ou `setDateHeader` est déjà défini, la nouvelle valeur écrase la première.
- Si l'en-tête a plusieurs valeurs, il faut utiliser l'une des méthodes `addHeader`, `addIntHeader` ou `addDateHeader` pour les ajouter.
- IL faut toujours définir les en-têtes HTTP de réponse avant d'écrire le corps de la réponse.

Quelques en-têtes HTTP de réponse

Content-Type

- L'en-tête `Content-Type` permet de définir le format des données de la réponse avec la méthode `setHeader`. Mais il est préférable d'utiliser la méthode `setContentType(String type)`.

- Exemple

```
response.setContentType("image/jpeg");
// est préférée à
response.setHeader("Content-Type", "image/jpeg");
```

Quelques en-têtes HTTP de réponse

WWW-Authenticate

- L'en-tête **`WWW-Authenticate`** permet de dire au navigateur de :
  - afficher une boite de login pour demander le login et le mot de passe de l'utilisateur,
  - relancer la requête avec l'en-tête http **`Authorization`** contenant le login et le mot de passe l'utilisateur codés en base64.

- Exemple :

```
response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
response.setHeader("WWW-Authenticate",
                   "Basic realm=Authentifiez-vous");
```

Quelques en-têtes HTTP de réponse

Refresh

- L'en-tête HTTP **`Refresh`** permet de demander au navigateur de rafraichir la page affichée ou d'afficher une autre après une certaine période d'attente (en secondes).

- Exemple

```
// Rafraichir la page après 5 secondes
response.setIntHeader("Refresh", 5);

// Afficher la page spécifiée avec URL après 10 secondes
response.setHeader("Refresh", "10; URL=http://...");
```

Quelques en-têtes HTTP de réponse

Content-Disposition

- L'en-tête **`Content-Disposition`** permet d'informer le client que la réponse est un fichier attaché et l'action à exécuter (l'afficher ou le télécharger).

- Exemple

```
// Afficher le fichier
response.setHeader("Content-Disposition",
                   "inline; filename=fichier.pdf");

// Proposer le téléchargement
response.setHeader("Content-Disposition",
                   "attachment; filename=fichier.pdf");
```

Quelques en-têtes HTTP de réponse

Content-Length

- Indique la taille de la réponse en octets. Il est utile pour le navigateur lorsque la réponse est un fichier attaché.

- Exemple

```
// Ecrire les octets d'un fichier dans un ByteArrayOutputStream
ByteArrayOutputStream baos = new ByteArrayOutputStream();
baos.write(octets);

// Indiquer la taille de la réponse au client
response.setContentLength(baos.size());

// Ecrire le contenu du ByteArrayOutputStream dans la réponse
ServletOutputStream out = response.getOutputStream();
out.write(baos.toByteArray());
```

Quelques en-têtes HTTP de réponse

Last-Modified

- Permet d'indiquer au navigateur la date de création de la réponse.
- Est utilisé conjointement avec l'en-tête de requête `If-Modified-Since` pour vérifier si la réponse a changé depuis son dernier envoi.
- L'implémentation de la méthode `getLastModified` de la classe `HttpServlet` permet de définir la valeur de cet en-tête.

Quelques en-têtes HTTP de réponse

- Exemple avec **getLastModified** :

```
@Override
protected long getLastModified(HttpServletRequest request) {
    return (new Date()).getTime();;
}
```

- Exemple sans **getLastModified** :

```
response.setDateHeader("Last-Modified", (new Date()).getTime(););
```

Quelques en-têtes HTTP de réponse

Cache-Control

- Permet d'indiquer au navigateur le comportement à adopter pour la mise en cache de la réponse.
- Quelques valeurs de l'en-tête :
  - `no-cache` : indique au navigateur que la réponse ne doit pas être mis en cache.
  - `no-store` : indique au navigateur que la réponse ne doit pas être mis en cache ou être enregistré par un serveur proxy.
  - `max-age=nbSecondes` : indique la durée de validité d'une réponse mis en cache. Le navigateur doit au-delà de `nbSecondes` demander une nouvelle réponse.

Quelques en-têtes HTTP de réponse

Cache-Control

- Quelques valeurs de l'en-tête (suite) :
  - **must-revalidate** : indique au navigateur de vérifier systématiquement la validité de la réponse qui est dans le cache avant de l'afficher.
  - **public** : indique que la réponse peut être mis en cache par le client ou par un cache partagé (serveur proxy).
  - **private** : indique que seul le client a l'autorisation de mettre la réponse en cache.
- Exemple :

```
response.setHeader("Cache-Control", "private");
response.addHeader("Cache-Control", "max-age=3600");
response.addHeader("Cache-Control", "must-revalidate");
```

Quelques types MIME [1] de réponse

Type	Signification
text/html	Document Html
text/plain	texte brute
application/msword	Document Microsoft Word
application/pdf	Fichier acrobat (.pdf)
application/vnd.ms-excel	Feuille Excel
application/vnd.ms-powerpoint	Presentation Powerpoint
image/jpeg	Image jpeg
image/png	Image png
video/mpeg	Video MPEG
audio/x-wav	Fichier de son Microsoft Windows

1. Multipurpose Internet Mail Extension

Inclusion de réponse

- La réponse d'une ressource (servlet, page JSP ou HTML) peut être incluse dans celle d'une autre.
- La réponse incluse ne peut ni modifier le code de statut ni les en-têtes de réponse.
- L'inclusion de réponse peut se faire à l'aide :
  d'un objet `javas.servlet.RequestDispatcher`
  ou
  de l'action standard `jsp:include`

Inclusion de réponse

Exemple 1 : un servlet ou une page JSP importe la réponse d'une ressource

```
// Dans le servlet ou la page JSP
RequestDispatcher rd = request.getRequestDispatcher("urlRessource");
rd.include(request, response);
```

Exemple 2 : une page JSP importe la réponse d'une ressource

```
// Dans la page JSP
<jsp:include page="urlRessource"/>
```

Inclusion de réponse

- Il est possible d'inclure des paramètres de requête pour la génération de la réponse à inclure.

Exemple 1 : avec `RequestDispatcher`

```
// Dans le servlet ou la page JSP
RequestDispatcher rd =
    request.getRequestDispatcher("urlRessource?param1=valeur1&...");
rd.include(request, response);
```

Exemple 2 : Avec l'action standard `jsp:include`

```
// Dans la page JSP
<jsp:include page="urlRessource" >
    <jsp:param name="param1" value="valeur1"/>
    ...
</jsp:include>
```